OPEN TRACK PS

PS1: Real-Time Financial Crisis Early Warning System

Problem

Economic crises — bank collapses, liquidity crashes, sovereign defaults — are detected too late. By the time traditional indicators surface instability, the damage is already compounding. Regulators, institutions, and markets are perpetually reactive, never anticipatory. There is no system that continuously monitors the full spectrum of global financial signals and raises an alarm before a crisis becomes a catastrophe.

What to build

A real-time financial crisis early warning platform that ingests global macroeconomic and market data to predict banking instability, market crashes, and liquidity shortages before they materialize. The system should continuously monitor cross-market signals — including interbank lending rates, sovereign bond spreads, foreign exchange volatility, derivatives exposure, and institutional capital flows — and apply predictive modeling to detect early-stage stress patterns that precede systemic failure. It should identify correlations across geographies and asset classes that human analysts would miss at scale and speed. The platform should surface probabilistic risk scores across three crisis categories — banking instability, market crashes, and liquidity shortages — with transparent reasoning behind each alert, so analysts can act on warnings rather than dismiss them as black-box outputs. Given the high uncertainty inherent in macroeconomic forecasting, the system should be honest about confidence intervals, flag when signal quality degrades, and continuously recalibrate its models against unfolding real-world outcomes to improve predictive accuracy over time.

OPEN TRACK PS

PS2: Academic Integrity Analyzer

Problem

Academic plagiarism has evolved beyond direct copying. A growing pattern involves students composing submissions by splicing sections from multiple existing papers, creating a superficially original document that evades traditional plagiarism detection tools. These composite papers are difficult to catch because no single section is copied verbatim — the fraud lies in the assembly. Build a system that detects this form of academic dishonesty through stylometric analysis, citation forensics, and semantic source tracing.

What to build

A document integrity analysis tool that ingests a research paper PDF and produces a structured forensic report assessing the likelihood of stitched authorship. The system should extract and segment the paper by paragraph, then perform writing style analysis across sections to identify statistically significant shifts in voice, vocabulary distribution, sentence structure, and linguistic fingerprint. Sections exhibiting distinct stylometric profiles should be flagged as potentially authored by different individuals. In parallel, the system should conduct citation graph analysis — examining whether reference clusters are temporally or thematically inconsistent across sections, which is a strong signal of independently sourced content being assembled post-hoc. The core detection mechanism should apply stylometry to estimate the minimum number of distinct authors present in the document. Results should be presented as a paragraph-level authorship heatmap, visually mapping each section to its inferred style cluster. Flagged sections should then be cross-referenced against arxiv and IEEE databases via semantic search to surface probable source papers, giving examiners traceable evidence rather than probabilistic suspicion alone.

OPEN TRACK PS

PS3: Repository Architecture Navigator

Problem

Onboarding a new developer onto a large codebase is one of the most expensive and disruptive processes in software engineering. Without a structural map of the system, newcomers spend weeks manually tracing imports, reading hundreds of files, and repeatedly interrupting senior engineers just to understand how the pieces fit together. There is currently no tool that automatically extracts and visualizes the true architecture of a codebase from the code itself.

What to build

A developer productivity tool that ingests a GitHub repository URL, performs static code analysis, and renders an interactive visual architecture graph that makes the system immediately understandable to anyone new to the codebase. The system should extract module relationships, file dependencies, call graphs, and data flow paths from the source code, then present this as a navigable graph where developers can click any file to see what it depends on and what depends on it. Entry points, core business logic, utility layers, and external integrations should be automatically identified and visually distinguished. Every component in the graph should carry an AI-generated summary explaining its purpose in plain language. High-impact files that affect multiple services should be highlighted to signal where changes carry the most risk. The system should also generate a recommended onboarding path — an ordered list of files a new developer should read first to reach a working understanding of the system as fast as possible. Optionally, the system should analyze commit history to show how the architecture evolved over time, detect undocumented or orphaned modules that add cognitive load without contributing functionality, and support natural language queries such as "where is authentication handled?" or "show the payment flow" — returning highlighted subgraphs as answers.

OPEN TRACK PS

PS4: AI-Powered Live Meeting Intelligence System

Problem

Meetings are among the most time-intensive activities in any organization, yet the output — decisions made, actions assigned, follow-ups committed to — is routinely lost to poor note-taking, selective memory, and lack of accountability. Teams repeatedly discuss the same unresolved topics across multiple meetings, follow-ups fall through the cracks, and no single participant has an accurate record of what was agreed. There is no tool that passively captures the full intelligence of a meeting in real time and converts it into structured, actionable output without any manual effort.

What to build

A real-time meeting intelligence system that runs as a background companion during any live call — Zoom, Google Meet, or Microsoft Teams — requiring no change to existing workflows. The system should hook into system audio to transcribe the meeting on the fly, automatically detecting action items as they are spoken and assigning them to the participant who was mentioned. Simultaneously, it should monitor the screen and capture timestamped screenshots whenever a significant visual — a slide, diagram, or shared document — appears, preserving context that transcription alone cannot capture. During the call, a live sidebar should display a rolling summary and a dynamic to-do list that updates continuously as the conversation progresses, giving all participants a shared real-time record of where the meeting stands. The moment the call ends, the system should push a structured debrief to every participant containing decisions made, open questions, a screenshot gallery, and each person's personalized action item digest — so individuals receive only what is relevant to them. It should auto-generate calendar invites for any follow-up meetings mentioned conversationally, and sync assigned tasks directly to Jira, Linear, or Notion. Across sessions, the system should maintain a cross-meeting debt log that tracks topics which are repeatedly discussed but never resolved — surfacing recurring blockers that leadership can act on before they compound.

OPEN TRACK PS

PS5: Developer Career Intelligence System

Problem

Developers consistently misjudge their own skill level. Resumes are written from self-perception rather than demonstrated ability, portfolios are curated to hide weaknesses, and career advice remains generic to the point of uselessness. A developer who thinks they are mid-level may have junior-grade authentication code in production. A developer who claims full-stack proficiency may have never written a tested, modular backend in their life. There is currently no tool that audits what a developer has actually built, compares it honestly against the market, and tells them exactly what stands between them and the next level.

What to build

A developer career intelligence platform that ingests a GitHub profile, project links, or a live application URL and produces a brutally honest 360-degree audit of demonstrated skill versus claimed experience. The system should pull the developer's complete GitHub activity and analyze code quality across every repository — measuring modularity, test coverage, documentation completeness, naming conventions, and architectural patterns. For live applications, it should run automated UI/UX audits checking responsiveness across screen sizes, accessibility scores, load times, animation smoothness, and interaction feedback quality. Reading directly from the codebase, it should surface over-engineered solutions, security anti-patterns, poor database design, unoptimized queries, and copied code the author does not demonstrably understand. After the full audit, the system should cross-reference the developer's real skill level against current job postings — identifying not just which roles they qualify for today, but which specific companies, locally and remotely, would realistically hire them right now. It should place the developer in a percentile ranking among those with comparable experience, and identify precisely which skills represent the gap to the next salary bracket. Feedback must be specific and traceable: not "learn system design," but "your authentication implementation in ProjectX has these three flaws that a senior engineer would reject in code review — fixing this moves you from junior to mid-level." The system should then generate a personalized 90-day learning roadmap that attacks the developer's weakest points in order of career ROI, identifies which projects to lead with on their resume versus which ones are actively damaging their profile, and rewrites their resume bullet points based on what was actually found in the code — not what they claimed they did.

OPEN TRACK PS

PS6: Interview Rejection Analysis System

Problem

Candidates walk out of failed interviews with no idea what went wrong. Feedback from recruiters is either withheld entirely or too vague to act on, leaving candidates to repeat the same mistakes across every subsequent interview. There is no tool that objectively analyzes what actually happened inside an interview and tells candidates precisely where they lost the opportunity.

What to build

A post-interview debrief platform where candidates upload or record their interview sessions to receive a detailed breakdown of why they likely did not progress. The system should analyze speech patterns such as filler word frequency, pauses, and pacing, evaluate answer structure using frameworks like clarity, depth, and relevance, and detect confidence drops through voice modulation and hesitation at specific questions. It should map each response against ideal expectations for that role, highlighting where answers lacked impact, specificity, or problem-solving depth, and identify patterns like vague storytelling, weak technical explanations, or misaligned behavioral responses. The tool should then provide precise, actionable feedback on what went wrong at each moment, rank weaknesses by impact on hiring decisions, and generate a targeted improvement plan to help candidates refine their responses and increase their chances in future interviews.